

AI が天気×時刻に合わせてプレイリスト作成

24FI009

井澤知紀

1.初めに

最近音楽を聴く機会が多くいろんな曲をプレイリストに追加していたが、今の気分や状況にあった曲が流れるまで曲をスキップすることが多かった。そこで、天気や時刻に合わせて聞く音楽を選んでプレイリストを作れば、その時の気分にあった曲にすぐたどり着けるのではないかと考えた。

2. プログラムで使用するデータとデータ記述言語

本プログラムでは、以下の技術要素を用いている。

- **Web API:** HTTP 通信を用いて外部サーバーからデータを取得する技術。本システムでは、気象庁、Spotify、Google Gemini の 3 つの API をマッシュアップしている。

- **JSON (JavaScript Object Notation):** 軽量なデータ交換フォーマット。API のレスポンスや Gemini へのリクエスト構造として使用している。

3. プログラムの設計と実装

3.1 実現した機能

ユーザーの Spotify アカウントからお気に入りアーティスト上位 10 名を特定し、その代表曲リストを取得。その後、選択された地域の天候と現在時刻を取得し、生成 AI (Gemini) がその状況に最もふさわしい 15 曲を厳選する。

使用方法

- 1.プログラムを実行し、ブラウザで Spotify の認証を行う。
- 2.地域を選択する。
- 3.AI が選んだ選曲リストが表示される
- 4.内容を確認し、「y」を入力すると Spotify 上に説明文付きのプレイリストが自動生成される。

3.2 system が利用するリソース

①気象庁 防災情報 XML(JSON 形式)

- ・ 概要: 日本各地の最新天気予報データ
- ・ 利用目的: 現在地の天候の取得
- ・ URL: <https://www.jma.go.jp/bosai/forecast/data/forecast/{areaCode}.json>
- ・ 形式: Web API (JSON)
- ・ 構造: timeSeries 配列内の weathers から現在の天候文字列を抽出

②Spotify Web API

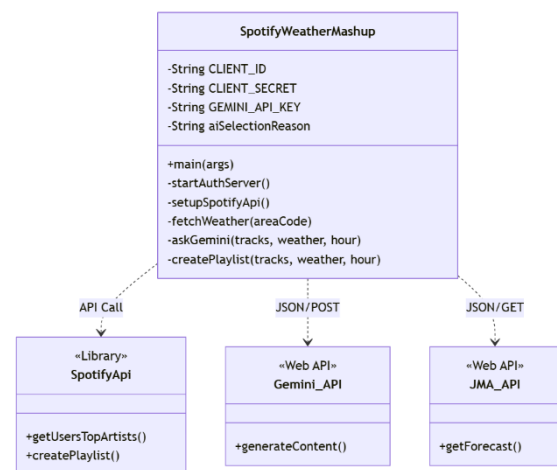
- ・ 概要: 楽曲データ、ユーザー情報、プレイリスト操作を提供する API
- ・ 利用目的: ユーザーの好みの取得、楽曲情報の収集、プレイリスト作成
- ・ URL: <https://api.spotify.com/v1>
- ・ 形式: Web API (JSON)
- ・ パラメータ: limit=10 (トップアーティスト取得数)、プレイリスト名、楽曲 URI 等

③Google Gemini API

- ・ 概要: 生成 AI
- ・ 利用目的: 天候、時間、楽曲リストを統合的に判断し、選曲と解説文を生成
- ・ URL: <https://generativelanguage.googleapis.com/v1beta/models/gemini-3-flash-preview:generateContent>
- ・ 形式: Web API (JSON)
- ・ 構造: contents/parts/text にプロンプトを送信

3.3 プログラムの構成

本プログラムは、外部の 3 つの API をマッシュアップし、ユーザーインターフェースとしてのコンソール入出力、および認可のためのローカルサーバー機能を統合した構成となっている。各コンポーネントの役割は次のようになっている。



クラス図

1.SpotifyWeatherMashup (メインクラス): プログラムの中核を担う。各 API との通信管理、ユーザーからの地域選択の受け付け、および AI からのレスポンス解析（正規表現を用いた ID 抽出）を行う。

2.SpotifyApi (外部ライブラリ): se.michaelthelin.spotify ライブラリを使用。OAuth 2.0 認証フローの管理、ユーザーの視聴履歴の取得、および新規プレイリストの作成と楽曲追加を制御する。

3.JMA_API (気象庁 API): 指定された地域コードに基づき、最新の気象予報データを JSON 形式で提供する。本プログラムでは「現在の天候」を特定するために利用する。

4.Gemini_API (生成 AI API): Google が提供する最新の gemini-3-flash-preview モデルを利用。天候、時刻、楽曲リストをコンテキストとして受け取り、人間のような自然な選曲理由の作成と、状況に即した楽曲の選別を行う。

クラス間の関係

メインクラスが各外部 API を呼び出す構造となっている。特に askGemini メソッドは、気象庁 API から得た環境情報と Spotify API から得たユーザーの好みを統合し、その結果を createPlaylist メソッドに渡すことで最終的なアウトプットを生成する。

3.4 データ構造とアルゴリズム

- ・ **データ構造:** 取得した楽曲情報を List<Track> で管理。AI からの回答は String として受け取り、正規表現を用いて 22 文字の Spotify ID を抽出するアルゴリズムを実装した。

- ・ **マッシュアップの工夫**

1. 予報データの最初の要素のみを取得し split することで数時間後の予報に左右されない今の天気の特化させた。

2. AI に対し、解説文と ID リストを同時に出力させ、文字の位置から両者を分離・抽出するアルゴリズムを構築した。

3.5 その他工夫点

- ・ Spotify API にレコメンド機能や自分のよく聞く曲を持ってくるリクエストを発見したが、開発者モードのため使用できないものや、ダミーデータを返すものなどあったため、このようなよく聞くアーティスト情報を取得し AI に選曲させる形にした。

- ・ プレイリストを自動で作成する前に選曲リストを表示し、ユーザーが納得した場合にのみ作成する内容にした

4. 実験

4.1 実行条件

入力データ	内容	データの規模・範囲
ユーザーの視聴傾向	Spotify API 経由で取得した個人の Top Artists	上位 10 名のアーティストを選出。各アーティストから 10 曲、計 100 曲を候補母集団として入力。
気象データ	気象庁 API から取得した地域別予報	実行時点の最新予報のデータ。3 時間ごとの時系列データのうち、最も現在時刻に近い物を抽出。
時間情報	システムクロックから取得した現在時刻	0~23 字の整数値。AI が朝・昼・夜・深夜を判定するための文脈として使用。

表 1:入力データとその概要

外部ライブラリ

- ・ Spotify SDK se.michaelthelin.spotify:spotify-web-api-java
- ・ org.json:json (ver.20231013)

外部ツール

- ・ Google Gemini API (モデル: gemini-3-flash-preview / v1beta)
- ・ Spotify Web API
- ・ 気象庁 防災情報 JSON

4.2 実験結果

プログラムを実行するとブラウザが開き、Spotify のログイン画面へと飛ぶ。今回はログインした情報が残っているため、以下のような画面になっている(図 1)。同意すると Eclipse に戻るようなメッセージが表示されるので、Eclipse に戻るとコンソールに以下のような分が表示されている(図 2)。表示されている地域の中から自分の現在地点に近い場所を選ぶとお勧めの曲が選別され、選曲リストが表示される(図 3)。プレイリストを作成するか聞かれるので y または n を入力する。n を入力した場合はプログラムが終了し、y を入力した場合は Spotify 上に図 4 のようなプレイリストが作成される。



Spotifyが以下に接続できるようにします

weatherSpotify

ご本人様でない場合

お客さまは weatherSpotify に以下の
事項を許可したものとみなされます。

👁️ アカウントのデータをみる

- 氏名、ユーザー名、プロフィール画像、Spotifyでのフォロワー数、公開プレイリスト。

📊 アクティビティをみる

- あなたのトップアーティストとコンテンツ

📖 アカウント上での Spotify の操作

- プレイリストの作成、編集、フォロー
- プレイリストの作成、編集、フォロー

同意する

キャンセル

アクセス権はアカウント設定からいつでも削除できます。

図 1:同意画面

```
SpotifyWeatherMashup [Java アプリケーション] C:\pleiades\2025-03\java\17\bin\javaw.exe (2
ブラウザで承認してください...
```

```
=== 1. 場所の番号を選んでください (1.東京 2.大阪 3.札幌 4.名古屋 5.福岡) ===
```

図 2:承認後のコンソール

```
SpotifyWeatherMashup [Java アプリケーション] C:\pleiades\2025-03\java\17\bin\javaw.exe (202
ブラウザで承認してください...
```

```
=== 1. 場所の番号を選んでください (1.東京 2.大阪 3.札幌 4.名古屋 5.福岡) ===
```

```
1
```

```
現在の状況: 晴れ (1時)
```

```
=== 2. あなたの好きなアーティスト10人の楽曲リストを作成中... ===
```

```
=== 3. Gemini AI が今の天気と時間に合う曲を厳選中... ===
```

```
--- AIが選んだ選曲リスト ---
```

```
1: 前前前世 - movie ver. / RADWIMPS
```

```
2: 君と羊と青 / RADWIMPS
```

```
3: お気に召すまま / Eve
```

```
4: メトロシティ / imase
```

```
5: 晴る / ヨルシカ
```

```
6: ただ君に晴れ / ヨルシカ
```

```
7: 怪獣の花唄 / Vaundy
```

```
8: 花占い / Vaundy
```

```
9: ピースサイン - Peace Sign / Kenshi Yonezu
```

```
10: さよならまたいつか! - Sayonara / Kenshi Yonezu
```

```
11: 感電 / Kenshi Yonezu
```

```
12: ミスター・ブルースカイ / Macaroni Empitsu
```

```
13: 洗濯機と君とラヂオ / Macaroni Empitsu
```

```
14: シーグラス / Saucy Dog
```

```
15: 分かってないよ / WurtS
```

```
この内容でプレイリストを作成しますか? (y/n):
```

図 3:選曲リストの表示



図 4:作成されたプレイリスト

5. 考察

5.1 機能

もともとは Spotify API のレコメンド機能も使ったものが作りたかったが、結果的に AI の作成したプレイリストの質が非常に高かったため、単なる機械的な選曲以上のものになったと感じる。改良案として、現在の気温データや季節を考慮するようにする、AI が渡された曲を分析し、そのユーザーが好きそうな曲も候補に加えるなどの機能を追加してみてもいいのではと考えた。

5.2 実現方法

本プログラムでは、外部 API との通信、データ処理、UI(コンソール)を一つのクラスに集約した。小規模なマッシュアップとして見通しが良く、適切な構造であったと考える。データ構造については Spotify から取得した楽曲を `List<Track>` で保持し、AI へのプロンプト生成時に一括して文字列化する手法をとった。これにより、メモリ効率を維持しつつ、AI に対して 100 曲という十分な母集団をコンテキストとして提供することが可能となった。

マッシュアップの核心となる選曲アルゴリズムにおいて、生成 AI を採用したことは非常に有効であった。レコメンド機能などのリクエストが使用できず、Spotify API の検索機能(単純にタイトルに晴がつく曲を探す等)では不可能な曲調や歌詞の世界観などといった抽象的な概念と現在の天気・時刻を紐づけることができた。

6.おわりに

本演習を通じ、複数の API を組み合わせることで、単体では実現不可能な新しいユーザー体験を創出できることを学んだ。今後は、さらに多くのユーザーが手軽に利用できる Web アプリケーション形式への拡張を目指したい。

感想

Spotify API の中で使えないリクエストが多い、OAuth2.0 認証できないなどの細かいトラブルに何度もぶつかり Spotify API を使うのをやめて別のものにしようかと考えましたが、一つずつログを解析して使えるものだけでどうにか自分の作りたいものが作れないか試行錯誤していく過程が楽しかったです。

参考文献

・ Spotify API の基礎知識と仕様解説から利用方法やデータ分析まで最新活用事例付き完全ガイド 著:ハウスケアラボ運営局

[SpotifyAPI の基礎知識と仕様解説から利用方法やデータ分析まで最新活用事例付き完全ガイド - ハウスケアラボ](#)